Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



Prof. Dr. Sascha Alda
Moritz Balg, Javed Razzaq, Luca Ringhausen und Team

13. Oktober 2026

# Übungen zur Vorlesung Software Engineering I
# WS 2026 / 2027

**Übungsblatt Nr. 3**
(Abgabe Mittwoch, den 21. Oktober 2026, **09:00 Uhr**)

**Aufgabe 1 (Modellierung Scrum-Prozess mit Onboarding, 10 Punkte):**

Ein IT-Dienstleister entwickelt für einen Kunden Individualsoftware im Rahmen eines Projekts. Das Projekt wird nach Scrum durchgeführt. Das Scrum-Team besteht aus vier Entwicklern, einem Product Owner und einem Scrum Master.

Zu Beginn eines neuen Sprints werden immer mal wieder neue Softwareentwickler eingestellt, die ab sofort in dem Projekt mitarbeiten sollen. Diese neuen Mitarbeiter arbeiten seit einigen Jahren zwar für den IT-Dienstleister, haben aber weder mit dem Kunden noch mit der projektspezifischen Softwarearchitektur, den Entwicklungswerkzeugen, der verwendeten Software (inklusive der notwendigen Lizenzen) und den Arbeitsabläufen des Teams und Kunden gearbeitet.

Das Team möchte die neuen Mitarbeiter möglichst schnell integrieren, ohne die laufende Sprint-Arbeit und die Zusammenarbeit mit dem Kunden zu beeinträchtigen. Man ist nun an einem Prozessmodell interessiert, um auch für zukünftige Neu-Einstellungen von Mitarbeitern einen flexiblen Onboarding-Prozess zu haben. Dieser Onboarding-Prozess soll in einem Scrum-Prozess integriert werden.

Modellieren Sie den Ablauf eines *einfachen* Scrum-Prozesses unter Berücksichtigung des Onboardings des neuen Mitarbeiters als UML-Aktivitätsdiagramm. Berücksichtigen Sie die wesentlichen Aktivitäten und Ereignisse von Scrum (vgl. Kapitel 3, Folie 42). Modellieren Sie den Scrum-Prozess nicht zu detailliert, sondern beschränken Sie sich auf wesentliche Aspekte. Anbei ein paar hilfreiche Fragenstellungen für das Onboarding, die Sie gerne berücksichtigen *können*:

- Welche Onboarding-Aktivitäten können *parallel* zur regulären Sprint-Arbeit stattfinden? Welche in einer *Sequenz*?
- Wie wird entschieden, wann er eine Aufgabe selbstständig übernehmen kann?
- Welche Ressourcen muss ein neuer Mitarbeiter zu Beginn des Projekts erhalten?

Berücksichtigen Sie in Ihrem Modell auch das Dokument „Produkt Backlog“. Swim Lanes *können* Sie bei Bedarf gerne berücksichtigen.

**Aufgabe 2 (Weiterentwicklung Container mit diversen Mustern. 20 Punkte)**

In dieser Aufgabe soll die Klasse `Container` aus dem Übungsblatt 2 weiterentwickelt werden. Falls sie diese Aufgabe noch nicht gelöst haben, dann können sie auf die Musterlösung der Übung 2 zurückgreifen (ab Mittwochabend online verfügbar). Als interne Datenstruktur soll eine Liste verwendet werden, die mit einem Typ `Member` als Generic parametrisiert ist. Folgende Änderungen (engl.: *change request* (*CR*)) sind dabei zu berücksichtigen:

*CR1*: Es soll zur Laufzeit zugesichert werden, dass von der Klasse `Container` nur *ein einziges Mal ein Objekt* erzeugt werden kann und somit nur ein Objekt davon im Speicher existiert. Wie lässt sich das realisieren, welches Pattern ist hier relevant? Hinweis: beachten sie in ihrem Lösungsweg die Verwendung einer *statischen* Methode zur Erzeugung des `Container`-Objekts. Erweitern sie die Klasse `Container` entsprechend. Weitere Hinweise dazu in der Übung.

*CR2*: Die Klasse `Container` soll um eine Methode `store` erweitert werden, welche die aktuell in einem `Container`-Objekt hinzugefügten `Member`-Objekte *persistent* auf einen Datenspeicher abspeichert. Die Methode hat folgende Signatur:

```
+ store() : void {throws PersistenceException}
```

Auch das Laden von `Member`-Objekten z.B. nach einem Neustart soll mit der Methode `load` möglich sein. Diese neue Methode der Klasse `Container` hat die Signatur:

```
+ load() : void {throws PersistenceException}
```

Befinden sich zum Zeitpunkt des Ladens `Member`-Objekte in der Liste, so sollen diese einfach überschrieben werden.

Von außerhalb eines Containers muss eine zugehörige Persistenz-Strategie in den Container flexibel *gesetzt* werden können. Eine Persistenz-Strategie muss das Interface `PersistenceStrategy` implementieren und die dort vorgegebenen Methoden überschreiben. Zudem muss über das Generic der Type definiert werden, der hier abgespeichert werden soll (hier: `Member`). Das Interface sowie die dazugehörigen Klassen (u.a. die Klasse `PersistenceException)` können sie aus dem GitLab-Repository der Vorlesung beziehen:

https://git.fslab.de/salda2m/se1ws26

Dort finden sie auch zwei Implementierungen des Interface `PersistenceStrategy` im Package `org.hbrs.se.ws26.exercises.uebung3.persistence`: die Klasse `Persistence-StrategyStream` sowie die Klasse `PersistenceStrategyMongoDB.`

Die Implementierung für MongoDB wird *noch* nicht unterstützt, ein Aufruf der Methoden wirft eine Exception. Eine Implementierung wird hier nicht erwartet von ihnen. Jedoch sollen sie das mögliche Werfen der Exception mit einem JUnit5-Test testen (siehe *CR4*).

Die Implementierung für Stream sollten sie in dieser Aufgabe hinzufügen. Zum Speichern muss die Klasse `ObjectOutputStream` aus dem Package `java.io` verwendet werden (bitte eigenständig recherchieren!). Zum Laden muss die Klasse Klasse `ObjectInputStream` aus dem Package `java.io` verwendet werden (bitte eigenständig recherchieren!). Einige Hinweise und gute Links (!) finden sie in den Kommentaren im Source Code auf GitHub.

*CR3*: Refaktorisieren Sie den Source Code der Klasse `Container`, indem sie Methode `dump` zur Ausgabe von Member-Objekten aus der Klasse `Container` in eine neue Klasse `MemberView` bewegen (engl. *move*). Eine Ausgabe soll nur noch in der `MemberView`-Klasse über diese „bewegte“ Methode erfolgen, die von der `MemberView`-Klasse bereitgestellt wird mit einer neuen Signatur:

```
+ dump( liste : List<Member> )
```

Um die aktuelle Liste von Member-Objekten zu erhalten, sollte eine entsprechende Methode im `Container` bereitgestellt werden, bitte diese auch implementieren:

```
+ getCurrentList() : List<Member>
```

Geben sie auch eine Implementierung einer `Client`-Klasse an, welche diverse Member-Objekte erzeugt, in den Container hinzufügt, diese wieder mittels der Methode `getCurrentList` ausliest und dann in die Klasse `MemberView` zur Ausgabe übergibt. Achtung: Aus dieser `Client`-Klasse und auch aus der `MemberView`-Klasse darf keine Persistenz-Strategie in den Container gesetzt werden! Hinweis: Realisieren sie eine `Main`-Klasse, welche die notwendigen Instanziierungen und Initialisierungen durchführt.

Frage: Welches Pattern zur Implementierung und Bereitstellung der Persistenz-Strategien wird hier angewandt? Hinweise finden sich auch im Source Code ;-)

Modellieren Sie auch ein UML-Klassendiagramm der resultierenden Klassen (Hinweis: dieses Modell wird in den Übungen entwickelt).

*CR4*: Entwickeln sie einen JUnit5-Testfall mit mindestens 4 aussagekräftigen Test-Methoden, um ihre Klasse Container *hinreichend* zu testen. Folgende Testfälle sollten dabei zumindest getestet werden (jeder Testfall eine Test-Methode):

- Keine Strategie von außen gesetzt („Test auf NULL“)
- Verwendung der Strategie „`PersistenceStrategyMongoDB`“

- Fehlerhafte Location des Files, in dem Streams ihre Objekte abspeichern (Tipp: FileStreams mögen keine Directories. Die Location kann für den Test über eine Methode

  ```
  setLocation ( location : String )
  ```

  (Objektmethode in Klasse `PersistenceStrategyStream)` gerne gesetzt werden.

- „Round-Trip-Test“: Alle Member-Objekte im Singleton löschen, Objekt hinzufügen, Liste persistent abspeichern, Objekt aus Container löschen und Liste wieder einladen. Nach jedem Schritt den Zustand des Containers testen! Für das Löschen aller Member-Objekte können Sie eine Methode

  ```
  deleteAllMembers()
  ```

  in den Container implementieren (Tipp: intern in der Liste die Methode `clear()` aufrufen)

Auf dem GitLab-Repository finden Sie eine Test-Klasse, die Ihnen einige Vorgaben und Hinweise für diese Tests gibt.